

Complete Specification and Design for Simile

Anthony Hunt

May 30, 2026

Contents

1	Overview	3
1.1	High Level Ideas	3
2	Motivation	3
3	Language	3
3.1	Lexicon	4
3.1.1	Whitespace	4
3.1.2	Identifiers	4
3.1.3	Primitives	4
3.1.4	Expressions	4
3.1.5	Reserved Keywords	6
3.2	Syntax	7
3.3	Static Analysis - Type System	9
3.3.1	Base Environment Interactions	10
3.3.2	Subtype Interactions	10
3.3.3	The Set Type	11
3.3.4	Primitive Types	11
3.3.5	Collection Types	11
3.3.6	Relation Subtypes	12
3.3.7	Simple Commands	13
3.3.8	Quantification Expressions	14
3.3.9	Boolean Operations	14
3.3.10	Equality and Membership	15
3.3.11	Set Operations	15
3.3.12	Bag Operations	16
3.3.13	Relation Operations	16
3.3.14	Relational Subtypes After Operations	16
3.3.15	Sequence Operations	17
3.3.16	Numerical Operations	17
3.3.17	Record Types	18
3.3.18	Compound Commands	18
3.3.19	Built-in Functions	19

3.4	Static Analysis - Type System Extensions with Traits	20
3.4.1	Trait-trait interactions	20
3.4.2	Type-trait interactions	21
3.5	Definedness	24
3.6	Language Examples	24
4	Features and Behaviours	24
5	User-Facing Specification	24
5.1	(Abstract) Data Types	24
5.1.1	Primitives	24
5.1.2	Sets	25
5.1.3	Relations	26
5.1.4	Bags	27
5.1.5	Sequences	28
5.2	Data Type Operations	28
5.2.1	Primitives	28
5.2.2	Sets	28
5.2.3	Relations	28
5.2.4	Bags	28
5.2.5	Sequences	28
6	Implementation Specification	28
6.1	Abstract Data Types	28
6.2	Atomic Operations	28
6.3	Optimization Frontend	28
6.4	Optimization Backends	28
7	AST Lowering and Optimization Rules	28
7.1	Syntactic Sugar for Bags	28
7.2	Syntactic Sugar for Sequences	29
7.3	Builtin Functions	29
7.4	Comprehension Construction	29
7.5	Disjunctive Normal Form	30
7.6	Or-wrapping	31
7.7	Generator Selection	31
7.8	GSP to Loop	32
7.9	Relational Subtyping Loop Simplification	33
7.10	Loop Code Generation	34
7.11	Replace and Simplify	35

1 Overview

This document serves as a living specification of the underlying term rewriting system used in the compiler for a modelling-focused programming language. It includes an overview of the language Simile, a user-facing reference, and implementation plans.

1.1 High Level Ideas

General Strategy A basic strategy to optimize set and relational expressions is:

1. Normalize the expression as a set comprehensions
2. Simplify and reorganize conjuncts of the set comprehension body

Intuition The TRS for this language primarily involves lowering collection data type expressions into pointwise boolean quantifications. Breaking down each operation into set builder notation enables a few key actions:

- Quantifications over sets ($\{x \cdot G \mid P\}$) are naturally separated into generators (G) and (non-generating) predicates (P). For sets, at least one membership operator per top-level conjunction in G will serve as a concrete element generator in generated code. Then, top level disjunctions will select one membership operation to act as a generator, relegating all others to the predicate level. For example, if the rewrite system observes an intersection of the form $\{x \cdot x \in S \wedge x \in T\}$, the set construction operation must iterate over at least one of S and T . Then, the other will act as a condition to check every iteration (becoming $\{x \cdot x \in S \mid x \in T\}$).
- By definition of generators in quantification notation, operations in G must be statements of the form $x \in S$, where x is used in the “element” portion of the set construction. Statements like $x \notin T$ or checking a property $p(x)$ must act like conditions since they do not produce any iterable elements.
- Any boolean expression for conditions may be rewritten as a combination of $\neg$, $\vee$, and $\wedge$ expressions. Therefore, by converting all set notation down into boolean notation and then generating code based on set constructor booleans, we can accommodate any form of predicate function.

2 Motivation

3 Language

This section denotes the design and behaviour of Simile.

Much of the language design is inherited from Event-B and the mathematical language definition in `[metayerEventBMathLang`

3.1 Lexicon

Since Simile makes extensive use of Event-B-like set notation and otherwise non-ASCII characters, this section contains a list of such characters, translations to ASCII symbols where suitable, and other useful lexing information.

3.1.1 Whitespace

Similar to Python, indentation and newline tokens are significant in Simile. In particular, nested code blocks, as in if-statements, for-loops, and procedure definitions, require the use of indentations to signify the beginning and end. Indentation/newlines may also be used within collection-type comprehensions or enumerations, similar to the style seen in JSON. Comments are single-lined and start with a pound symbol (#).

3.1.2 Identifiers

Identifiers in Simile follow the standard regular expression, using only ASCII letters, underscores, and numbers: $[a-zA-Z_][a-zA-Z_0-9]^*$.

3.1.3 Primitives

Integers and floating point numbers follow a similar style to modern programming languages. The dash character (–) may be used in either subtraction or as a unary negation. Superscript numbers may be used for constant exponentiation. Booleans (True/False) follow Python style, with capitalized first letters. Strings are delimited with double-quotes (") and may be multilined.

3.1.4 Expressions

ASCII tokens will be provided as an alternative to unicode format where suitable.

The following are symbols used for expressions:

Token	ASCII Token	Name
.	.	Center Dot (Quantifier Separator) ¹
.	.	Dot
,	,	Comma
:	:	Colon
;	;	Semicolon
		Vertical Bar
λ	lambda	Lambda
(	(	Left Parenthesis
)	)	Right Parenthesis
<	<<	Left Bracket
	[	Left Bracket (Alternative)
>	>>	Right Bracket
	]	Right Bracket (Alternative)
{	{	Left Brace
}	}	Right Brace
[[	[[	Left Double Bracket
]]	]]	Right Double Bracket
=	=	Equals
≠	!=	Not Equals
¬	not	Not

¹ Anywhere . is valid, . is valid.

Token	ASCII Token	Name
	!	Not (Alternative)
$\wedge$	and	And
$\vee$	or	Or
$\Rightarrow$	=>	Implies
$\equiv$	==	Equivalent
$\neq$	!=	Not Equivalent
$\forall$	forall	For All
$\exists$	exists	There Exists
	+	Plus
	-	Minus
	*	Multiplication
	/	Division
	div	Integer Division
	mod	Modulo
	^	Exponent
$<$		Less Than
$\leq$	<=	Less Than or Equal
$>$		Greater Than
$\geq$	>=	Greater Than or Equal
$\in$	in	In (Membership)
$\notin$	not in	Not In (Membership)
$\cup$	\/	Union
$\cap$	/\	Intersection
$\backslash$	\	Backslash (Set Minus)
$\times$	><	Cartesian Product
$\subset$	<<:	Subset
$\subseteq$	<:	Subset or Equal
$\supset$	:>>	Superset
$\supseteq$	:>	Superset or Equal
$\not\subset$	!<<:	Not Subset
$\not\subseteq$	!<:	Not Subset or Equal
$\not\supset$	!:>>	Not Superset
$\not\supseteq$	!:>	Not Superset or Equal
$\bigcup$	union	General Union
$\bigcap$	intersection	General Intersection
$\mapsto$	->	Maplet
$\oplus$	<+>	Relation Overriding
$\circ$	circ	Composition
++	++	Concatenation
-1	~	Inverse
$\triangleleft$	<	Domain Restriction
$\triangleleft\!\!\triangleleft$	<<	Domain Subtraction
$\triangleright$	>	Range Restriction
$\triangleright\!\!\triangleright$	>>	Range Subtraction
$\leftrightarrow$	<->	Relation

Token	ASCII Token	Name
$\Leftrightarrow$	<<->	Total Relation ²
$\Leftarrow$	<->>	Surjective Relation ³
$\Leftrightarrow$	<<->>	Total Surjective Relation ⁴
$\mapsto$	+->	Partial Function
$\rightarrow$	-->	Total Function
$\hookrightarrow$	>+>	Partial Injection
$\rightarrow$	>->	Total Injection
$\twoheadrightarrow$	+->>	Partial Surjection
$\twoheadrightarrow$	-->>	Total Surjection
$\xrightarrow{\quad}$	>->>	Bijection
$\dots$		Upto
$\mathbb{P}$	powerset	Powerset
$\mathbb{P}_1$	powerset1	Non-Empty Powerset

The following are symbols used for programming constructs:

(Unicode) Token	(ASCII) Token	Name
$\coloneqq$	:=	Assignment
$\in$	::	Choice (Non-deterministic Membership) Assignment
$\rightarrow$	->	Right Arrow

3.1.5 Reserved Keywords

The following list of symbols and identifiers are reserved for programming constructs:

true	false	if	else	for
while	record	enum	procedure	is
is not	return	break	continue	from
import	skip	with	type	

And for built-in types:

int	float	string	bool	$\mathbb{Z}$	$\mathbb{N}$
$\mathbb{N}_1$	set	sequence	bag	relation	generic
tuple					

And built-in procedures:

min	map_min	max	map_max	choice	dom
ran	card	size	sum	cast	cast_with
print					

²Brave Sans Mono unicode: U+E100

³Brave Sans Mono unicode: U+E101

⁴Brave Sans Mono unicode: U+E102

3.2 Syntax

We follow the standard EBNF notation for describing the language structure.

Start ::= Statements

Statements ::= {SimpleStmt | CompoundStmt}

SimpleStmt ::= Expr [[':' Expr] (':=' | ':∈') Expr] NEWLINE [WithStmt]
| ControlFlowStmt NEWLINE
| ImportStmt NEWLINE

WithStmt ::= INDENT 'with' Expr NEWLINE {Expr NEWLINE} DEDENT

ControlFlowStmt ::= 'return' [Expr]
| 'break'
| 'continue'
| 'skip'

ImportStmt ::= 'import' STRING
| 'from' STRING 'import' ImportList

ImportList ::= '*'
| FlatTupleIdentifier

FlatTupleIdentifier ::= IDENTIFIER {' , ' IDENTIFIER }
| '(' IDENTIFIER {' , ' IDENTIFIER } ')'

Expr ::= Quantification | Predicate

Quantification ::= 'λ' [FlatTupleIdentifier] '·' Predicate '∣' Expr
| '∪' QuantificationBody
| '∩' QuantificationBody

QuantificationBody ::= IdentList '·' Predicate '∣' Expr
| Expr '∣' Predicate

IdentList ::= IdentListItem {' , ' IdentListItem}

IdentListItem ::= IDENTIFIER
| IdentListItem '↦' IdentListItem
| '(' IdentList ')'

Predicate ::= UnquantifiedPredicate
| '∀' IdentList '·' Predicate
| '∃' IdentList '·' Predicate

UnquantifiedPredicate ::= Implication
| UnquantifiedPredicate ('≡' | '≠') Implication

Implication ::= Disjunction
 | Disjunction { '⇒' Disjunction }

Disjunction ::= Conjunction { '∨' Conjunction }

Conjunction ::= Negation { '∧' Negation }

Negation ::= AtomBool
 | '¬' Negation

AtomBool ::= PairExpr [('=' | '≠' | '<' | '>' | '≤' | '≥' | '∈' | '∉' | '⊂' | '⊆' | '⊃' | '⊇' | '⊄' | '⊈' | '⊅' | '⊆' | '⊇') PairExpr]

PairExpr ::= RelSetExpr
 | PairExpr '↦' RelSetExpr

RelSetExpr ::= SetExpr [('↔' | '↔' | '↔' | '↔' | '↔' | '→' | '↗' | '↘' | '↙' | '↘' | '↗' | '↘') RelSetExpr]

SetExpr ::= IntervalExpr
 | IntervalExpr { '∪' IntervalExpr }
 | IntervalExpr { '×' IntervalExpr }
 | IntervalExpr { '⊕' IntervalExpr }
 | IntervalExpr { '∘' IntervalExpr }
 | IntervalExpr { '∩' IntervalExpr }
 | IntervalExpr { '++' IntervalExpr }
 | IntervalExpr ('≪' | '≻') IntervalExpr { '∩' IntervalExpr } [RelSubExpr]

RelSubExpr ::= ('∖' | '▷' | '▷') IntervalExpr

IntervalExpr ::= ArithmeticExpr ['.' ArithmeticExpr]

ArithmeticExpr ::= Term
 | ArithmeticExpr ('+' | '-') Term

Term ::= Factor
 | Term ('*' | '/' | 'div' | 'mod') Factor

Factor ::= [('+' | '-')] Power

Power ::= Primary ['^' Factor | ['-'] ('INTEGER' | 'FLOAT')]

Primary ::= Atom
 | Atom ' ' IDENTIFIER
 | Atom '(' Expr { ',' Expr } ')'
 | Atom '[' Expr ']'

Atom ::= INTEGER | FLOAT | STRING | BOOLEAN | IDENTIFIER
 | Set | Sequence | Bag | Tuple
 | 'ℙ' '(' Expr ')' | 'ℙ₁' '(' Expr ')' | '(' Expr ')'

$\text{Set} ::= \text{'\{'} \text{ CollectionBody } \text{'\}'}$
 $\text{Bag} ::= \text{'\['} \text{ CollectionBody } \text{'\]'}$
 $\text{Sequence} ::= \text{'\langle'} \text{ CollectionBody } \text{'\rangle'}$
 $\text{Tuple} ::= \text{'('} \text{'\)' } \mid \text{'('} [\text{NEWLINE INDENT}] \text{Expr} \text{'\,'} [\text{NEWLINE [INDENT]}] [\text{EnumerationBody}] \text{'\)'}$
 $\text{CollectionBody} ::= \text{QuantificationBody} \mid \text{EnumerationBody}$
 $\text{EnumerationBody} ::= [\text{NEWLINE INDENT}] \text{Expr} \{ \text{'\,'} [\text{NEWLINE [INDENT]}] \text{Expr} \} [\text{NEWLINE DEDENT}]$
 $\text{CompoundStmt} ::= \text{IfStmt} \mid \text{ForStmt} \mid \text{WhileStmt}$
 $\quad \mid \text{RecordStmt} \mid \text{ProcedureStmt}$
 $\text{IfStmt} ::= \text{'if'} \text{ Predicate } \text{'\:'} \text{ Block } [\text{ElseStmt}]$
 $\text{ElseStmt} ::= \text{'else \:'} \text{ Block}$
 $\quad \mid \text{'else if'} \text{ Predicate } \text{'\:'} \text{ Block } [\text{ElseStmt}]$
 $\text{ForStmt} ::= \text{'for'} \text{ IdentList } \text{'\in'} \text{ Expr } \text{'\:'} \text{ Block}$
 $\text{WhileStmt} ::= \text{'while'} \text{ Expr } \text{'\:'} \text{ Block}$
 $\text{RecordStmt} ::= \text{'record'} \text{ IDENTIFIER } \text{'\:'} \text{ NEWLINE INDENT } ($
 $\quad \text{'skip'} \mid \text{TypedName} \{ \text{'\,'} [\text{NEWLINE}] \mid \text{NEWLINE} \} \text{TypedName } \}$
 $\quad) \text{ NEWLINE DEDENT}$
 $\text{ProcedureStmt} ::= \text{'procedure'} \text{ IDENTIFIER } \text{'('} \text{ TypedName } \{ \text{'\,'} \text{TypedName } \} \text{'\)' } \text{'\rightarrow'} \text{ Expr } \text{'\:'} \text{ Block}$
 $\text{TypedName} ::= \text{IDENTIFIER} [\text{'\:'} \text{Expr}]$
 $\text{Block} ::= \text{SimpleStmt}$
 $\quad \mid \text{NEWLINE INDENT} \text{ Statements } \text{DEDENT}$

3.3 Static Analysis - Type System

Let Γ be the type environment set consisting of elements either *identifier* : *Type* or *Type*. The typing rules have been organized in distinct categories for readability. The notation *Type*[*x, y, z*] denotes refinements on *Type* either through instantiating parametric (generic) types or restricting values by way of **with** clauses.

Type refinements may be applicable to types with certain traits (ex., min/max values for ordered types, definedness of expressions with certain values, set sizes).

3.3.1 Base Environment Interactions

Includes: Mechanisms for adding/extracting variables from the type environment.

$$\begin{array}{c}
\overline{\emptyset \vdash \diamond} \quad (\text{Env } \emptyset) \\
\frac{\Gamma \vdash A \quad I \notin \text{dom}(\Gamma)}{\Gamma, I : A \vdash \diamond} \quad (\text{Env } I) \\
\frac{\Gamma_1, I : A, \Gamma_2 \vdash \diamond}{\Gamma_1, I : A, \Gamma_2 \vdash I : A} \quad (\text{Fetch Identifier})
\end{array}$$

3.3.2 Subtype Interactions

Includes: Generic subtyping rules.

$$\begin{array}{c}
\frac{\Gamma \vdash A}{\Gamma \vdash A \leq A} \quad (\text{Reflexive Subtype}) \\
\frac{\Gamma \vdash A \leq B \quad \Gamma \vdash B \leq C}{\Gamma \vdash A \leq C} \quad (\text{Transitive Subtype}) \\
\frac{\Gamma \vdash a : A \quad \Gamma \vdash A \leq B}{\Gamma \vdash a : B} \quad (\text{Subsumption}) \\
\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{Any}} \quad (\text{Top Type}) \\
\frac{\Gamma \vdash A}{\Gamma \vdash A \leq \text{Any}} \quad (\text{Sub Top Type}) \\
\frac{\Gamma \vdash A' \leq A \quad \Gamma \vdash B \leq B'}{\Gamma \vdash A \rightarrow B \leq A' \rightarrow B'} \quad (\text{Sub Function}) \\
\frac{\Gamma \vdash A \leq B}{\Gamma \vdash \text{set}[A] \leq \text{set}[B]} \quad (\text{Sub Set}) \\
\frac{\Gamma \vdash A_1 \leq B_1 \quad \Gamma \vdash A_2 \leq B_2}{\Gamma \vdash A_1 \times A_2 \leq B_1 \times B_2} \quad (\text{Sub Product}) \\
\frac{\Gamma \vdash A_1 \leq B_1 \dots A_n \leq B_n \quad \Gamma \vdash A_{n+1} \dots A_{n+m}}{\Gamma \vdash \text{Record}[I_1 : A_1, \dots, I_n : A_n, I_{n+1} : A_{n+1}, \dots, I_{n+m} : A_{n+m}] \leq \text{Record}[I_1 : B_1, \dots, I_n : B_n]} \quad (\text{Sub Record}) \\
\frac{\Gamma \vdash T}{\Gamma \vdash T[\text{with } x] \leq T} \quad (\text{Type Refinement})
\end{array}$$

3.3.3 The Set Type

Includes: Universal type; everything is a set. The empty set is a part of every type (aside from primitives).

$$\begin{array}{c}
\frac{\Gamma \vdash T}{\Gamma \vdash \text{set}[T]} \quad (\text{Powerset}) \\
\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{EmptySet}} \quad (\text{Emptyset})
\end{array}$$

$$\frac{\Gamma \vdash \text{EmptySet}, \text{set}[T]}{\text{EmptySet} \leq \text{set}[T]} \quad (\text{Emptyset Bottom})$$

$$\frac{\Gamma \vdash e_1, \dots, e_n : A}{\Gamma \vdash \{e_1, \dots, e_n\} : \text{set}[A]} \quad (\text{Set Enumeration})$$

3.3.4 Primitive Types

Includes: Basic types built in to the language, like numbers (int and float), strings, and booleans. Although these could all be represented as sets, it is useful to distinguish these common types from other collections or user-defined types.

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{bool}} \quad (\text{Primitives - bool})$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{int}} \quad (\text{Primitives - int})$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{float}} \quad (\text{Primitives - float})$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{str}} \quad (\text{Primitives - str})$$

$$\frac{\Gamma \vdash \text{str} \quad \Gamma \vdash \text{int}}{\Gamma \vdash \text{nat} \leq \text{int}[\text{with } \text{min} = 0]} \quad (\text{Nat from int})$$

$$\frac{\Gamma \vdash \text{int}, \text{float}}{\text{int} \leq \text{float}} \quad (\text{Int from float})$$

3.3.5 Collection Types

Includes: Sequences, relations, enums (frozen sets), and bags (multisets); Syntactic sugar for abstract data types defined in terms of sets.

$$\frac{\Gamma \vdash T_1, \dots, T_n}{\Gamma \vdash \text{tuple}[T_1, \dots, T_n] \quad \text{tuple}[T_1, \dots, T_n] = T_1 \times \dots \times T_n} \quad (\text{Tuples})$$

$$\frac{\Gamma \vdash \diamond}{\Gamma \vdash () : \text{EmptyTuple}} \quad (\text{Empty Tuple})$$

$$\frac{\Gamma \vdash \text{EmptyTuple}, T}{\Gamma \vdash \text{EmptyTuple} \leq \text{tuple}[T]} \quad (\text{Empty Tuple Bottom})$$

$$\frac{\Gamma \vdash e_1 : A_1, \dots, e_n : A_n}{\Gamma \vdash (e_1, \dots, e_n) : \text{tuple}[A_1, \dots, A_n]} \quad (\text{Tuple Enumeration})$$

$$\frac{\Gamma \vdash \text{set}[T_1], \text{set}[T_2]}{\Gamma \vdash T_1 \leftrightarrow T_2 \quad T_1 \leftrightarrow T_2 = \text{set}[\text{tuple}[T_1, T_2]]} \quad (\text{Relation Type})$$

$$\frac{\Gamma \vdash T, \text{nat}}{\Gamma \vdash \text{bag}[T] \leq T \rightarrow \text{nat}} \quad (\text{Bag Type})$$

$$\frac{\Gamma \vdash e_1, \dots, e_n : A}{\Gamma \vdash \llbracket e_1, \dots, e_n \rrbracket : \text{bag}[A]} \quad (\text{Bag Enumeration})$$

$$\begin{array}{c}
\frac{\Gamma \vdash T, nat}{\Gamma \vdash sequence[T] \leq nat \rightarrow T} \quad \text{(Sequence Type)} \\
\frac{\Gamma \vdash e_1, \dots, e_n : A}{\Gamma \vdash \langle e_1, \dots, e_n \rangle : sequence[A]} \quad \text{(Sequence Enumeration)} \\
\frac{\Gamma \vdash s_1, \dots, s_n : str \quad A = \{s_1, \dots, s_n\} \quad immutable(A)}{\Gamma \vdash enum[A] \leq set[A, with immutable]} \quad \text{(Enum from static set)}
\end{array}$$

3.3.6 Relation Subtypes

Includes: Relation subtypes examine four key properties of relations: totality (t), surjectivity (t_r), one-to-manyness ($1-$), and many-to-oneness ($1-r$). Syntactic sugar to produce refined relations with these properties follows the definitions presented in Event-B. See Section 5.1.3 for more details.

$$\begin{array}{c}
\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \leftrightarrow T_2 \quad T_1 \leftrightarrow T_2 = (T_1 \leftrightarrow T_2)[with \ t]} \quad \text{(Relation Subtype - Total Relation)} \\
\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \leftrightarrow T_2 \quad T_1 \leftrightarrow T_2 = (T_1 \leftrightarrow T_2)[with \ t_r]} \quad \text{(Relation Subtype - Surjective Relation)} \\
\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \leftrightarrow T_2 \quad T_1 \leftrightarrow T_2 = (T_1 \leftrightarrow T_2)[with \ t, \ with \ t_r]} \quad \text{(Relation Subtype - Total Surjective Relation)} \\
\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrow T_2 \quad T_1 \rightarrow T_2 = (T_1 \leftrightarrow T_2)[with \ 1-]} \quad \text{(Relation Subtype - Partial Function)} \\
\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrow T_2 \quad T_1 \rightarrow T_2 = (T_1 \leftrightarrow T_2)[with \ t, \ with \ 1-]} \quad \text{(Relation Subtype - Total Function)} \\
\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrow T_2 \quad T_1 \rightarrow T_2 = (T_1 \leftrightarrow T_2)[with \ 1-, \ with \ 1-r]} \quad \text{(Relation Subtype - Partial Injection)} \\
\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrow T_2 \quad T_1 \rightarrow T_2 = (T_1 \leftrightarrow T_2)[with \ t, \ with \ 1-, \ with \ 1-r]} \quad \text{(Relation Subtype - Total Injection)} \\
\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrow T_2 \quad T_1 \rightarrow T_2 = (T_1 \leftrightarrow T_2)[with \ t_r, \ with \ 1-]} \quad \text{(Relation Subtype - Partial Surjection)} \\
\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrow T_2 \quad T_1 \rightarrow T_2 = (T_1 \leftrightarrow T_2)[with \ t, \ with \ t_r, \ with \ 1-]} \quad \text{(Relation Subtype - Total Surjection)} \\
\frac{\Gamma \vdash set[T_1], set[T_2]}{\Gamma \vdash T_1 \rightarrow T_2 \quad T_1 \rightarrow T_2 = (T_1 \leftrightarrow T_2)[with \ t, \ with \ t_r, \ with \ 1-, \ with \ 1-r]} \quad \text{(Relation Subtype - Bijection)}
\end{array}$$

3.3.7 Simple Commands

Includes: Assignment, type assignment, type refinements, simple programming language commands/statements.

$$\frac{\Gamma \vdash E : A \quad I \notin dom(\Gamma) \vee \Gamma \vdash I : A}{\Gamma \vdash I : A := E} \quad \text{(Variable Assignment)}$$

$\frac{\Gamma \vdash T \quad I \notin \text{dom}(\Gamma)}{\Gamma \vdash I := T}$	(Type Alias Assignment)
$\frac{\Gamma \vdash I := T}{\Gamma, I \vdash \diamond \quad I = T}$	(Type Alias)
$\frac{\Gamma \vdash E : A, E_1 : A_1, \dots, E_n : A_n \quad (I \notin \text{dom}(\Gamma) \vee \Gamma \vdash I : A)}{\Gamma \vdash I : A[\text{with } E_1 : A_1, \dots, \text{with } E_n : A_n] := E}$	(Refined Variable Assignment)
$\frac{\Gamma \vdash T, E_1 : A_1, \dots, E_n : A_n \quad I \notin \text{dom}(\Gamma)}{\Gamma \vdash I := T[\text{with } E_1 : A_1, \dots, \text{with } E_n : A_n]}$	(Refined Type Alias)
$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{break} : C}$	(Command - break)
$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{continue} : C}$	(Command - continue)
$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{skip} : C}$	(Command - skip)
$\frac{\Gamma \vdash E : A}{\Gamma \vdash (\text{return } E) : C}$	(Command - return)

3.3.8 Quantification Expressions

Includes: Set, sequence, relation, and bag comprehensions, in addition to (set based) lambda expressions. Comprehension variables may be bound through either an *Identifier* or an arbitrarily nested tuple of *Identifiers*

$\frac{\Gamma, I_1 : T_1, \dots, I_n : T_n \vdash E : A, P : \text{bool} \quad I_i \notin \text{dom}(\Gamma) \quad i \neq j \implies I_i \neq I_j}{\Gamma \vdash (\lambda I_1, \dots, I_n \cdot P \mid E) : (T_1, \dots, T_n) \rightarrow A}$	(Lambda Expression)
$\frac{I = x_1 : T_1, \dots, x_n : T_n \quad \Gamma \vdash \text{binds}(P, I) \quad \Gamma, I \vdash P : \text{bool}, E : A}{\Gamma \vdash (x_1, \dots, x_n \cdot P \mid E) : \text{QuantificationBody}[I, A]}$	(Quantification Body ⁵)
$\frac{\Gamma \vdash g : \text{set}[T] \quad \Gamma \vdash \text{structuralMatch}(I, T)}{\Gamma \vdash \text{binds}(I \in g, I)}$	(Binds (with generator))
$\frac{\Gamma \vdash \text{binds}(P_1, I), \dots, \text{binds}(P_n, I)}{\Gamma \vdash \text{binds}(\bigvee P_i, I)}$	(Binds with OR)
$\frac{\Gamma, I \vdash Q : \text{bool} \quad \Gamma \vdash \text{binds}(P, I)}{\Gamma \vdash \text{binds}(P \wedge Q, I)}$	(Binds with AND)
$\frac{\Gamma \vdash T \quad x \notin \text{dom}(\Gamma)}{\Gamma \vdash \text{structuralMatch}(x, T)}$	(Structural Match)
$\frac{\Gamma \vdash \text{structuralMatch}(x_1, T_1), \dots, \text{structuralMatch}(x_n, T_n)}{\Gamma \vdash \text{structuralMatch}((x_1, \dots, x_n), (T_1, \dots, T_n))}$	(Structural Match with Tuple)
$\frac{\Gamma \vdash (I \cdot P \mid E) : \text{QuantificationBody}[M, A]}{\Gamma \vdash (\bigcup I \cdot P \mid E) : A}$	(General Union)
$\frac{\Gamma \vdash (I \cdot P \mid E) : \text{QuantificationBody}[M, A]}{\Gamma \vdash (\bigcap I \cdot P \mid E) : A}$	(General Intersection)

$\frac{\Gamma \vdash (I \cdot P) : QuantificationBody[M, bool]}{\Gamma \vdash (\forall I \cdot P) : bool}$	(Forall)
$\frac{\Gamma \vdash (I \cdot P) : QuantificationBody[M, bool]}{\Gamma \vdash (\exists I \cdot P) : bool}$	(Exists)
$\frac{\Gamma \vdash (I \cdot P \mid E) : QuantificationBody[M, A]}{\Gamma \vdash \{I \cdot P \mid E\} : set[A]}$	(Set Comprehension)
$\frac{\Gamma \vdash (I \cdot P \mid E) : QuantificationBody[M, A]}{\Gamma \vdash \llbracket I \cdot P \mid E \rrbracket : bag[A]}$	(Bag Comprehension)
$\frac{\Gamma \vdash (I \cdot P \mid E) : QuantificationBody[M, A]}{\Gamma \vdash [I \cdot P \mid E] : sequence[A]}$	(Sequence Comprehension)

3.3.9 Boolean Operations

Includes: Common predicate logic operations (equivalence, implication, and, or, etc.).

Let $\otimes = \{\equiv, \neq, \implies, \wedge, \vee\}$

$\frac{\Gamma \vdash b_1, b_2 : bool}{\Gamma \vdash b_1 \otimes b_2 : bool}$	(Binary Boolean Operations)
$\frac{\Gamma \vdash b_1 : bool}{\Gamma \vdash \neg b_1 : bool}$	(Boolean Operations - Negation)

3.3.10 Equality and Membership

Includes: Equality and ordering comparisons, set membership.

Let $\otimes = \{=, \neq\}$

$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2}{\Gamma \vdash a_1 \otimes a_2 : bool}$	(Equals)
---	----------

Let $\otimes = \{<, \leq, >, \geq\}$

$\frac{\Gamma \vdash a_1, a_2 : T \quad T \in \{int, float, nat\}}{\Gamma \vdash a_1 \otimes a_2 : bool}$	(Ordering Operators)
---	----------------------

Let $\otimes = \{\in, \notin\}$

$\frac{\Gamma \vdash a_1 : T_1, a_2 : set[T_2]}{\Gamma \vdash a_1 \otimes a_2 : bool}$	(Set Membership)
--	------------------

3.3.11 Set Operations

Includes: Subset comparisons, union, intersection, cartesian product, etc. Also includes Maplet and Range.

Let $\otimes = \{\subset, \subseteq, \supset, \supseteq, \not\subset, \not\subseteq, \not\supset, \not\supseteq\}$

$$\frac{\Gamma \vdash a_1 : \text{set}[T_1], a_2 : \text{set}[T_2]}{\Gamma \vdash a_1 \otimes a_2 : \text{bool}}$$

(Set Ordering Operations)

Let $\otimes = \{\cup, \cap, \setminus\}$

$$\frac{\Gamma \vdash a_1, a_2 : \text{set}[T]}{\Gamma \vdash a_1 \otimes a_2 : \text{set}[T]}$$

(Set Operations)

$$\frac{\Gamma \vdash a_1 : \text{set}[T_1], a_2 : \text{set}[T_2]}{\Gamma \vdash a_1 \times a_2 : T_1 \leftrightarrow T_2}$$

(Cartesian Product)

$$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2}{\Gamma \vdash a_1 \mapsto a_2 : \text{tuple}[T_1, T_2]}$$

(Maplet)

$$\Gamma \vdash a_1, a_2 : \text{int}$$

(Numerical Range)

$$\frac{\Gamma \vdash a_1 \dots a_2 : \text{set}[\text{int}, \text{ with } \min = a_1, \text{ with } \max = a_2]}{\Gamma \vdash a : T}$$

$$\frac{\Gamma \vdash a : T}{\Gamma \vdash \mathbb{P}(a) : \text{set}[T]}$$

(Set Operations - Powerset)

3.3.12 Bag Operations

Includes: Bag union, bag intersection, etc.

Let $\otimes = \{\cup, \cap, +, -\}$

$$\frac{\Gamma \vdash a_1, a_2 : \text{bag}[T]}{\Gamma \vdash a_1 \otimes a_2 : \text{bag}[T]}$$

(Bag Operations)

$$\frac{\Gamma \vdash a : T_1 \leftrightarrow T_2, b : \text{bag}[T_1]}{\Gamma \vdash a[b] : \text{bag}[T_2] \quad a[b] = a^{-1} \circ b}$$

(Bag Operations - Image)

3.3.13 Relation Operations

Includes: Relation overriding, composition, domain/range manipulation. See Section 5.1.3 for changes in relational subtypes after applying operations.

$$\frac{\Gamma \vdash a : T_1 \leftrightarrow T_2, b : T_1}{\Gamma \vdash a(b) : T_2}$$

(Relation Operations - Function Call)

$$\frac{\Gamma \vdash a : T_1 \leftrightarrow T_2, b : \text{set}[T_1]}{\Gamma \vdash a[b] : \text{set}[T_2]}$$

(Relation Operations - Image)

$$\frac{\Gamma \vdash a_1, a_2 : T_1 \leftrightarrow T_2}{\Gamma \vdash a_1 \oplus a_2 : T_1 \leftrightarrow T_2}$$

(Relation Operations - Overriding)

$$\frac{\Gamma \vdash a_1 : T_1 \leftrightarrow T_2, a_2 : T_2 \leftrightarrow T_3}{\Gamma \vdash a_1 \circ a_2 : T_1 \leftrightarrow T_3}$$

(Relation Operations - Composition)

$$\frac{\Gamma \vdash a_1 : \text{set}[T_1], a_2 : T_1 \leftrightarrow T_2}{\Gamma \vdash a_1 \triangleleft a_2 : T_1 \leftrightarrow T_2}$$

(Relation Operations - Domain Restriction)

$\frac{\Gamma \vdash a_1 : \text{set}[T_1], a_2 : T_1 \leftrightarrow T_2}{\Gamma \vdash a_1 \triangleleft a_2 : T_1 \leftrightarrow T_2}$	(Relation Operations - Domain Subtraction)
$\frac{\Gamma \vdash a_1 : T_1 \leftrightarrow T_2, a_2 : \text{set}[T_2]}{\Gamma \vdash a_1 \triangleright a_2 : T_1 \leftrightarrow T_2}$	(Relation Operations - Range Restriction)
$\frac{\Gamma \vdash a_1 : T_1 \leftrightarrow T_2, a_2 : \text{set}[T_2]}{\Gamma \vdash a_1 \triangleright a_2 : T_1 \leftrightarrow T_2}$	(Relation Operations - Range Subtraction)

3.3.14 Relational Subtypes After Operations

Includes: Changes in relational subtyping (totality, surjectivity, etc.) after applying operations. For simplicity, we implement subtype changes in a few binary operators as a boolean mask representing: [with total (t), with total on range (t_r), with one-to-manyness ($1-$), with many-to-oneness ($1-r$)]. Subtyping is done conservatively: we keep subtype properties only when they are guaranteed. See Section 5.1.3 for more on relational subtyping.

$\frac{\Gamma \vdash s : \text{set}[T_1], r : (T_1 \leftrightarrow T_2)[\text{True}, x, y, z]}{\Gamma \vdash (s \triangleleft r) : (T_1 \leftrightarrow T_2)[\text{False}, x, y, z]}$	(Relational Subtype - Domain Restriction)
$\frac{\Gamma \vdash s : \text{set}[T_1], r : (T_1 \leftrightarrow T_2)[\text{True}, x, y, z]}{\Gamma \vdash (s \triangleleft r) : (T_1 \leftrightarrow T_2)[\text{False}, x, y, z]}$	(Relational Subtype - Domain Subtraction)
$\frac{\Gamma \vdash s : \text{set}[T_2], r : (T_1 \leftrightarrow T_2)[x, \text{True}, y, z]}{\Gamma \vdash (r \triangleright s) : (T_1 \leftrightarrow T_2)[x, \text{False}, y, z]}$	(Relational Subtype - Range Restriction)
$\frac{\Gamma \vdash s : \text{set}[T_2], r : (T_1 \leftrightarrow T_2)[x, \text{True}, y, z]}{\Gamma \vdash (r \triangleright s) : (T_1 \leftrightarrow T_2)[x, \text{False}, y, z]}$	(Relational Subtype - Range Subtraction)
$\frac{\Gamma \vdash r : (T_1 \leftrightarrow T_2)[w, x, y, z]}{\Gamma \vdash r^{-1} : (T_1 \leftrightarrow T_2)[x, w, y, z]}$	(Relational Subtype - Inverse)
$\frac{\Gamma \vdash r : (T_1 \leftrightarrow T_2)[a, b, c, d], t : (T_1 \leftrightarrow T_2)[w, x, y, z]}{\Gamma \vdash (r \oplus t) : (T_1 \leftrightarrow T_2)[w, x, c \wedge y, d \wedge z]}$	(Relational Subtype - Overriding)
$\frac{\Gamma \vdash r : (T_1 \leftrightarrow T_2)[a, b, c, d], t : (T_2 \leftrightarrow T_3)[w, x, y, z]}{\Gamma \vdash (r \circ t) : (T_1 \leftrightarrow T_3)[a, b \wedge x, c \wedge y, d \wedge z]}$	(Relational Subtype - Composition)

3.3.15 Sequence Operations

Includes: Concatenation

$\frac{\Gamma \vdash a_1, a_2 : \text{sequence}[T]}{\Gamma \vdash a_1 \uparrow a_2 : \text{sequence}[T]}$	(Sequence Operations - Concatenation)
---	---------------------------------------

3.3.16 Numerical Operations

Includes: Addition, subtraction, multiplication, etc. on natural numbers, integers, and floats.

$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{\text{int}, \text{nat}\}}{\Gamma \vdash a_1 \text{ div } a_2 : \max(T_1, T_2)}$	(Integer Operations - Division)
---	---------------------------------

$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{int, nat\}}{\Gamma \vdash a_1 \bmod a_2 : \max(T_1, T_2)}$	(Integer Operations - Modulo)
$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{int, nat, float\}}{\Gamma \vdash a_1 + a_2 : \max(T_1, T_2)}$	(Numerical Operations - Addition)
$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{int, nat, float\}}{\Gamma \vdash a_1 - a_2 : \max(T_1, T_2, int)}$	(Numerical Operations - Subtraction)
$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{int, nat, float\}}{\Gamma \vdash a_1 \div a_2 : float}$	(Numerical Operations - Floating Division)
$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{int, nat, float\}}{\Gamma \vdash a_1 * a_2 : \max(T_1, T_2)}$	(Numerical Operations - Multiplication)
$\frac{\Gamma \vdash a_1 : T \quad T \in \{int, nat, float\}}{\Gamma \vdash -a_1 : \max(int, T)}$	(Numerical Operations - Negation)
$\frac{\Gamma \vdash a_1 : T_1, a_2 : T_2 \quad T_i \in \{int, nat, float\}}{\Gamma \vdash a_1 \wedge a_2 : \max(T_1, T_2)}$	(Numerical Operations - Exponentiation)

3.3.17 Record Types

Includes: Types for named tuples.

$\frac{\Gamma \vdash a : Record[I : A, \dots]}{\Gamma \vdash a.I : A}$	(Records - Access)
$\frac{\Gamma \vdash A_1, \dots, A_n \quad \forall I_i, I_j \cdot I_i \neq I_j}{\Gamma \vdash Record[I_1 : A_1, \dots, I_n : A_n]}$	(Records - Type Definition)
$\frac{\Gamma \vdash a_1 : A_1, \dots, a_n : A_n}{\Gamma \vdash record(a_1 = A_1, \dots, a_n = A_n) : Record[a_1 : A_1, \dots, a_n : A_n]}$	(Records - Initialization)

3.3.18 Compound Commands

Includes: If, While, For, and Import statements, along with Procedure definitions and calls.

$\frac{\Gamma \vdash C_1, C_2}{\Gamma \vdash C_1 \backslash n C_2}$	(Command - Composition)
$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{import } M : C}$	(Command - Valid Import Module)
$\frac{\Gamma \vdash \diamond}{\Gamma \vdash \text{from } M \text{ import } I_1, \dots, I_n : C}$	(Command - Valid Import Names)
$\frac{\Gamma \vdash \text{import } M \text{ (with identifiers and types } I_1 : A_1, \dots, I_n : A_n)}{\Gamma, M = record(I_1 = A_1, \dots, I_n = A_n) \vdash \diamond}$	(Command - Import Module)
$\frac{\Gamma \vdash \text{from } M \text{ import } I_1, \dots, I_n \text{ (with types } A_1, \dots, A_n)}{\Gamma, I_1 : A_1, \dots, I_n : A_n \vdash \diamond}$	(Command - Import Names)

$\frac{\Gamma, I : \text{procedure}[A_1, \dots, A_n, R], x_1 : A_1, \dots, x_n : A_n \vdash C : R \quad I, x_1, \dots, x_n \notin \text{dom}(\Gamma) \quad i \neq j \implies x_i \neq x_j}{\Gamma \vdash \text{procedure } I = C : \text{procedure}[A_1, \dots, A_n, R]}$	(Command - Procedure Definition)
$\frac{\Gamma \vdash \text{procedure}[A_1, \dots, A_n, R], a_1 : A_1, \dots, a_n : A_n}{\Gamma \vdash I(a_1, \dots, a_n) : R}$	(Command - Procedure Call)
$\frac{\Gamma \vdash b : \text{bool}, \text{block } C}{\Gamma \vdash \text{if } b : C}$	(Command - If)
$\frac{\Gamma \vdash b : \text{bool}, \text{block } C_1, \text{block } C_2}{\Gamma \vdash \text{if } b : C_1 \text{ else } : C_2}$	(Command - If Else)
$\frac{\Gamma \vdash b : \text{bool}, \text{block } C}{\Gamma \vdash \text{while } b : C}$	(Command - While)
$\frac{\Gamma \vdash G : \text{set}[T] \quad \Gamma, i : T \vdash C \quad i \notin \text{dom}(\Gamma)}{\Gamma \vdash \text{for } i \in G : C}$	(Command - For)
$\frac{\Gamma \vdash D : \Gamma' \quad \Gamma' \vdash C}{\Gamma \vdash \text{block } D \setminus n C}$	(Command - Block)

3.3.19 Built-in Functions

Includes: Common set-related functions.

$\frac{\Gamma \vdash T[\text{with order} \in \text{traits}(T)]}{\Gamma \vdash \text{min} : \text{set}[T] \rightarrow T}$	(Built-in - Minimum)
$\frac{\Gamma \vdash T, \text{int}}{\Gamma \vdash \text{min} : \text{set}[T] \times (T \rightarrow \text{int}) \rightarrow T}$	(Built-in - Mapped Minimum)
$\frac{\Gamma \vdash T[\text{with order} \in \text{traits}(T)]}{\Gamma \vdash \text{max} : \text{set}[T] \rightarrow T}$	(Built-in - Maximum)
$\frac{\Gamma \vdash T, \text{int}}{\Gamma \vdash \text{max} : \text{set}[T] \times (T \rightarrow \text{int}) \rightarrow T}$	(Built-in - Mapped Maximum)
$\frac{\Gamma \vdash T}{\Gamma \vdash \text{choice} : \text{set}[T] \rightarrow T}$	(Built-in - Choice)
$\frac{\Gamma \vdash T}{\Gamma \vdash \text{dom} : T_1 \leftrightarrow T_2 \rightarrow \text{set}[T_1]}$	(Built-in - Domain)
$\frac{\Gamma \vdash T}{\Gamma \vdash \text{ran} : T_1 \leftrightarrow T_2 \rightarrow \text{set}[T_2]}$	(Built-in - Range)
$\frac{\Gamma \vdash T}{\Gamma \vdash \text{card} : \text{set}[T] \rightarrow \text{nat}}$	(Built-in - Cardinality)
$\frac{\Gamma \vdash T}{\Gamma \vdash \text{size} : \text{bag}[T] \rightarrow \text{nat}}$	(Built-in - Bag Size)
$\frac{\Gamma \vdash T \quad T \in \{\text{int}, \text{float}, \text{nat}\}}{\Gamma \vdash \text{sum} : \text{set}[T] \rightarrow T}$	(Built-in - Sum)

$$\begin{array}{c}
\frac{\Gamma \vdash T_1, T_2}{\Gamma \vdash \text{cast} : \text{type}[T_1] \times T_1 \rightarrow T_2} \quad \text{(Built-in - Cast)} \\
\frac{\Gamma \vdash T_1, T_2, x : E}{\Gamma \vdash \text{cast} : \text{type}[T_1] \times T_1 \times \text{set}[E] \rightarrow T_2[\text{with } x : E]} \quad \text{(Built-in - Cast With } ^6)
\end{array}$$

3.4 Static Analysis - Type System Extensions with Traits

3.4.1 Trait Collection

Traits are used to augment types, providing the compiler with additional context for the use of an abstract type that could influence the generated implementation. The following paragraphs list identified traits and provide justification for their existence.

Constant Pre-computation The type may contain values for easily identifiable constants, allowing for specialized generated instructions. When possible, constant traits will propagate along operations until the constant values can no longer be guaranteed.

- *Literal*: A trait containing a single literal value. If a type has this trait, it is guaranteed to be of this value.
- *Domain*: A looser form of *Literal* that carries a set of possible values. When applied, the type is restricted to (one of) the values from this *Domain*.
- Leave out for now: *UnknownElementInDomain*: Loosens the *Domain* guarantee so that values in the domain are only likely to appear, but not guaranteed? We would need to see if this opens up any optimizations...
- *Immutable*: Represents types whos values (and value contents) cannot be modified. Should we instead assume immutability and have a Mutable trait?

Arithmetic Restricts the value of primitive types. Collection types wishing to take advantage of this information should instead store it in the underlying primitive element type. For example, applying *Min* to a *set[int]* should be written as *set[int[with min = 0]]*, not *set[int, with min = 0]* (although we can loosen this with syntactic sugar). Like Event-B, *set* types are really powersets of the underlying element type, whereas primitive types represent values chosen from a *set* of primitive values.

- *Orderable*: Values of this type can be ordered. This may be useful for order-based implementations like BTrees for sets.
- *Min*: minimum value
- *Max*: maximum value

Collection Traits

- *Iterable*: All collections are iterable (they can be used as a for-loop generator).
- *Empty*: A collection with guaranteed cardinality = 0.
- *Size*: An over-estimate of a set's size. When available, Size will never under-estimate the number of elements in a collection.

- *UniqueElements*: All entries are guaranteed to be unique. More useful for the domain/range of relations, checking if a bag only has range = 1, or if a sequence has no repeats.
- *Total*: The elements in a set cover the entirety of the domain of the underlying element type. On relations, this means every possible pair combination is an element.

Relations Represents the relational subtype properties (injectiveness, surjectiveness, etc.)

- *TotalOnDomain*: All elements of the domain are within the mapping.
- *TotalOnRange*: All elements of the range are within the mapping.
- *ManyToOne*: Elements of the domain may be repeated with different range values.
- *OneToMany*: Elements of the range may be repeated with different range values. *ManyToOne* and *OneToMany* imply a one-to-one mapping

Generics

- *GenericBound*: Contains the set of types a generic may become. For example, $T[\text{with } GenericBound = \{int, float\}]$ represents any numeric value.

3.4.2 Trait-trait Interactions

Collection of trait-implies-trait interactions that are universal (i.e., they do not depend on a type). Most traits are really just boolean values, as in, does this type have this property. Thus when traits used with boolean operations should be interpreted as truthy/falsy.

Interpret these as if it were type initialization rules. So, if a type were initialized with these traits, we would get the following implications “for-free”. A type can only have one of each trait (except for multiple literals?), so traits given in the assumption can be used in the result without ambiguity or naming. Traits with function notation (parenthesis) are assumed to be new constructions.

TODO design decision: should traits be applied automatically or just verified? If we apply automatically, we could give best effort and then error/remove traits that are incompatible. In verification, we just throw an error. We may use trait restrictions per-type to further determine incompatibilities (ex. an integer cannot be empty?).

$\frac{Domain = \{\}}{\neg Domain}$	(Domain Empty)
$\frac{Literal \quad \neg Domain}{Domain := \{Literal\}}$	(Literal implies a Domain)
$\frac{Literal \quad Domain}{Domain := Domain \cup Literal}$	(Literal within Domain)
$\frac{Domain \quad Orderable \quad \neg Min}{Min := min(Domain)}$	(Orderable Domain without Min)
$\frac{Domain \quad Orderable \quad Min}{Min := min(Min, Domain)}$	(Orderable Domain with Min)
$\frac{Domain \quad Orderable \quad \neg Max}{Max = max(Domain)}$	(Orderable Domain without Max)

$\frac{Domain \quad Orderable \quad Max}{Max = max(Max, Domain)}$	(Orderable Domain with Max)
$\frac{Min}{Orderable}$	(Min implies Order)
$\frac{Max}{Orderable}$	(Max implies Order)
$\frac{UniqueElements \quad Size = len(Domain) \quad \neg Empty}{Total}$	(Full set)
$\frac{Size = 0}{Empty}$	(Empty Size)
$\frac{Size \neq 0}{\neg Empty}$	(Non-Empty Size)
$\frac{Size}{Iterable}$	(Size implies Iterable)
$\frac{Literal \quad Orderable \quad \neg Min}{Min = Literal}$	(Orderable Literal is Min)
$\frac{Literal \quad Orderable \quad \neg Max}{Max = Literal}$	(Orderable Literal is Max)

3.4.3 Type-trait interactions

Some types are incompatible with some traits, and some types must have some traits. This section lists the possible traits for each type - asterisks are for mandatory traits, and traits not listed are not allowed for that specific type.

All types Literal and Domain support constant propagations and compile time computations.

- Immutable
- Literals
- Domain

Bool

- Domain = {true, false}*

None

- Literal*
- Domain? (implied from literal)

String

- Empty
- Size (length)
- Unique?

- Iterable*
- Orderable (lexicographic)*

Int, Float

- Min
- Max
- Size (num of bits? or just depend on min/max? If so, min/max must be given, with a default value of 64 bit range)
- Orderable*

Tuple

- Min
- Max
- Size (length)
- UniqueElements
- Total
- Iterable*
- Empty
- Orderable (lexicographic if dependents are orderable?)

Maplet Tuple but with the addition of:

- Domain?
- Size = 2*

Record

- Iterable* (treat as a relation)
- Domain* (str entries)
- ManyToOne* (treat as a relation with keys = strings)

Procedure

- Domain should be stored in the domain of the input types since there are too many to count? Range should be the domain of the return type
- Need to worry about generics?

Set

- Orderable if children are
- Min
- Max
- Iterable*
- Empty
- UniqueElements*
- Size
- Total

Relation All traits except generic trait

Bag Same as relation, with ManyToOne*. Should we add a trait called Count, which is the sum of the range values?

Seq Same as relation, with ManyToOne*, Size*

Enum

1. Immutable*
2. Domain*
3. Literal*
4. Iterable*
5. UniqueElements*
6. Size*?
7. Can never be total unless enum size = 1, then always total

3.4.4 Operation-Trait Interactions

Operations on *Literal* traits will always produce a *Literal* where possible, following the effects of applying the function on values. For conciseness, we omit all *Literal* interactions and instead refer to the semantics of the operations themselves.

$$\frac{a : T[Max = x] \quad b : T[Min = y] \quad x < y}{(a < b) : Bool[Literal = true]} \quad \text{(Less than with max-min)}$$
$$\frac{a : T[Max = x] \quad b : T[Min = y] \quad x \leq y}{(a \leq b) : Bool[Literal = true]} \quad \text{(Less than or equal with max-min)}$$

$\frac{a : T[Min = x] \quad b : T[Max = y] \quad x > y}{(a < b) : Bool[Literal = false]}$	(Less than with min-max)
$\frac{a : T[Min = x] \quad b : T[Max = y] \quad x > y}{(a \leq b) : Bool[Literal = false]}$	(Less than or equal with min-max)
$\frac{a : Int[Literal] \quad b : Int[Literal]}{(a .. b) : set[Int[Min = a, Max = b], Literal = \{a, a + 1, ..., b\}, Total]}$	(Upto)
$\frac{s : set[Empty]}{(s = \{\}) : Bool[Literal = true]}$	(Is Empty)
$\frac{s : set[T, Size = x] \quad a : T}{(s \cup \{a\}) : set[T, Size = x + 1]}$	(Set add)
$\frac{s : set[T, Size = x, Total] \quad a : T}{(s \cup \{a\}) : set[T, Size = x, Total]}$	(Set add with Total)
$\frac{s : set[T, Size = x] \quad a : T}{(s \setminus \{a\}) : set[T, Size = x]}$	(Set delete)
$\frac{s : set[T, Size = x, Total] \quad a : T}{(s \setminus \{a\}) : set[T, Size = x - 1]}$	(Set delete removes Total)
$\frac{s : set[T[Domain = S]] \quad a : T[Literal = A]}{(a \in s) : Bool[Literal = A \in S]}$	(Membership within domain)
$\frac{s : set[T, Total] \quad a : T}{(a \in s) : Bool[Literal = true]}$	(Membership within total domain)
$\frac{a_1, a_2, ..., a_n : T}{\{a_1, a_2, ..., a_n\} : set[T, Size = n]}$	(Enumeration)
$\frac{s : set[T, Size = n]}{card(s) : Int[Max = n]}$	(Cardinality)
$\frac{s : set[T, Size = n, Total]}{powerset(s) : set[set[T], Size = 2^n, Total]}$	(Powerset)
$\frac{s : set[T, Empty]}{choice(s) : TypeError}$	(Empty choice fails (statically))
$\frac{s : set[T[Max = y, Size = n]]}{sum(s) : T[Max = y \times n]}$	(Sum bounds)
$\frac{s : set[T[Max = y, Size = n]]}{sum(s) : T[Max = y^n]}$	(Product bounds)
$\frac{s : set[T[Min = y]] \quad a : T[Literal = y] \quad a \in s}{min(s) : T[Literal = y]}$	(Min looks for Min)
$\frac{s : set[T[Max = y]] \quad a : T[Literal = y] \quad a \in s}{max(s) : T[Literal = y]}$	(Max looks for Max)

$\frac{s : \text{set}[T, \text{Literal} = a] \quad t : \text{set}[T, \text{Literal} = b]}{(s \cup t) : \text{set}[T, \text{Literal} = a \cup b]}$	(Union Literal)
$\frac{s : \text{set}[T[\text{Domain} = a]] \quad t : \text{set}[T[\text{Domain} = b]]}{(s \cup t) : \text{set}[T[\text{Domain} = a \cup b]]}$	(Union with widest Domain)
$\frac{s : \text{set}[T[\text{Min} = a]] \quad t : \text{set}[T[\text{Min} = b]] \quad a < b}{(s \cup t) : \text{set}[T[\text{Min} = a]]}$	(Union with lowest Min)
$\frac{s : \text{set}[T[\text{Max} = a]] \quad t : \text{set}[T[\text{Max} = b]] \quad a < b}{(s \cup t) : \text{set}[T[\text{Max} = b]]}$	(Union with highest Max)
$\frac{s : \text{set}[T, \text{Empty}] \quad t : \text{set}[T, x] \quad a < b}{(s \cup t) : \text{set}[T, x]}$	(Union with Empty keeps Traits)
$\frac{s : \text{set}[T, \text{Size} = x] \quad t : \text{set}[T, \text{Size} = y] \quad x < y}{(s \cup t) : \text{set}[T, \text{Size} = y]}$	(Union with widest size)
$\frac{s : \text{set}[T] \quad t : \text{set}[T, \text{Total}]}{(s \cup t) : \text{set}[T, \text{Total}]}$	(Union with Total stays)
$\frac{s : \text{set}[T[\text{GenericBound} = x]] \quad t : \text{set}[T[\text{GenericBound} = y]]}{(s \oplus t) : \text{set}[T[\text{GenericBound} = x \cap y]]}$	(Generics with set ops)
$\frac{s : \text{set}[T[\text{Domain} = x]] \quad t : \text{set}[V[\text{Domain} = y]]}{(s \times t) : \text{set}[\text{tuple}[T, V, \text{Domain} = x \times y]]}$	(Cartesian Product)
$\frac{s : \text{set}[T, \text{Total}] \quad t : \text{set}[V, \text{Total}]}{(s \times t) : \text{set}[\text{tuple}[T, V, \text{Total}]]}$	(Total Cartesian Product)
$\frac{s : \text{set}[T, \text{Min} = w, \text{Max} = x] \quad t : \text{set}[T, \text{Min} = y, \text{Max} = z] \quad w > y \quad x < z}{(s \subset t : \text{set}[T, \text{Min} = w, \text{Max} = x]) : \text{Bool}}$	(Subset with min-max)
$\frac{r : \text{relation}[T[\text{Domain} = x], V[\text{Domain} = y]]}{r^{-1} : \text{relation}[V[\text{Domain} = y], T[\text{Domain} = x]]}$	(Inverse swaps Domain)
$\frac{s : \text{relation}[T, V, \text{Empty}] \quad t : \text{relation}[V, U]}{s \circ t : \text{relation}[T, U, \text{Empty}]}$	(Empty Composition)
$\frac{r : \text{relation}[T, V] \quad s : \text{set}[T, \text{Empty}]}{r[s] : \text{set}[V, \text{Empty}]}$	(Image with Empty Arg)
$\frac{r : \text{relation}[T, V, \text{Empty}] \quad s : \text{set}[T]}{r[s] : \text{set}[V, \text{Empty}]}$	(Image with Empty Relation)
$\frac{r : \text{relation}[T, V, \text{TotalOnDomain}] \quad s : \text{set}[T, \text{Domain} = \text{domain}(r), \text{Size} > 0]}{r[s] : \text{set}[V, \text{Size} > 0]}$	(Image with ManyToOne)
$\frac{r : \text{relation}[T, V, \text{ManyToOne}] \quad s : \text{set}[T, \text{Size} = n]}{r[s] : \text{set}[V, \text{Size} = n]}$	(Image with ManyToOne)
$\frac{r : \text{relation}[T, V, \text{TotalOnDomain}]}{\text{domain}(r) : \text{set}[T, \text{Total}]}$	(Total domain)

$$\begin{array}{c}
\frac{r : \text{relation}[T, V, \text{TotalOnRange}]}{\text{range}(r) : \text{set}[V, \text{Total}]} \quad \text{(Total range)} \\
\frac{s : \text{sequence}[T, \text{Size} = x] \quad t : \text{sequence}[T, \text{Size} = y]}{s \mathbin{++} t : \text{sequence}[T, \text{Size} = x + y]} \quad \text{(Concat with Size)}
\end{array}$$

3.5 Definedness

3.6 Language Examples

4 Features and Behaviours

5 User-Facing Specification

5.1 (Abstract) Data Types

For the most part, Simile’s semantics resemble those of set theory and specification languages (specifically, Event-B). Aside from a few primitive types that abstract over bare set theory and programming language specific structures, like integer arithmetic and procedure definitions, all objects inherit properties from sets. For example, a sequence can be modelled as a set of pairs from natural numbers to the sequence element type.

In the forthcoming sections, we explicitly state the design of these set-theory-inspired types, useful properties, and underlying implementations.

5.1.1 Primitives

With consideration to execution efficiency and the extensive amount of existing optimization in modern languages, Simile makes use of the following basic types:

Booleans As the cornerstone of logical reasoning, booleans deserve an explicit type with reserved identifiers: $\mathbb{B} = \{\text{True}, \text{False}\}$. Further, Simile provides a healthy supply of common operations beyond $\wedge$ and $\vee$: $\implies$, $\equiv$, $\longleftarrow$, $\forall$, $\exists$, etc.

Integers While integers are a very mature type in the world of computing, trade-offs between arbitrary-precision and fixed-point arithmetic may require testing and careful review.

If Simile chooses to represent numbers through conventional fixed point arithmetic, struggles with overflow and extremely large numbers are compounded by the innate non-determinism within Simile. For example, calculating the sum of a set of very large numbers can be done in arbitrary order, according to the semantics of mathematics (and Simile). But in fixed-point arithmetic, the resulting value can very well differ based on the order of operations. In other words, commutative math operations are no longer commutative under fixed precision. At some point in the computation pipeline, Simile will need to decide the order in which to evaluate the set, which may cause hard-to-spot bugs and unexpected runtime failures.

Conversely, choosing arbitrary precision integers may bring Simile operators closer to mathematics at the cost of speed. The core features of Simile, while targeting formal methods users, attempt to efficiently execute set and relation operations. Adding checks for overflow on every mathematical operation may prove detrimental to performance.

The implementation decision essentially is one of context vs. safety. To benchmark Simile with these types, we gather a list of examples and judge whether the performance hit is worth the reliability benefits. Specific questions to examine: How often does overflow occur in these examples? Do numbers approach the upper limits of fixed

precision often enough to warrant accommodating infrastructure? What is the performance of fixed vs. arbitrary precision calculations.

Floating Point Numbers In a similar vein to integers, floats are an imperfect translation of real numbers into efficiently calculable representations. Since floats (or even reals) are not used as often in specifications as integers, we leave the implementation as the conventional 64-bit representation. However, we note that non-determinism on these operations will prove especially tricky, since the result of commutative complements may differ without explicit over/underflow.

Strings Strings are syntactic sugar over Sequences of characters, represented internally as a sequence of (fixed precision) integers. Eventually, Simile aims to support built-in string operations as seen in many common programming languages.

5.1.2 Sets

Foundational Simile type: an unordered, unique collection of objects.

Sets may be used in two ways:

1. Enumerations by way of $\{x, y, z\}$ (read: The set of elements x, y , and z)
2. Comprehensions of the form $\{x \cdot x \in S \mid f(x)\}$ (read: The set of function f applied to all elements in S). A small effort is made to accommodate a shorthand form, wherein $\{x \cdot x \in S \mid x\}$ may be written as $\{x \cdot x \in S\}$, but this syntax easily confuses bound/unbound variables and is not recommended for complex comprehension expressions.

Sets, as mathematical objects with only two major restrictions, lend themselves well to a variety of computational implementations. Arrays, ordered arrays, hash maps, bit sets, roaring bit sets, path maps, and tries can all be used to satisfy the uniqueness property and hide an internally ordered representation from users.

No matter the underlying implementation, the optimizer may make use of syntactically-analyzed set sizes to direct the choice of lowered iterator. Further, recording the cardinality of a set at runtime may prove beneficial to the performance of common cardinality-based specification expressions at a very small memory cost. Other properties based on element type, like max/min for numeric types, limits for numeric ranges, maximum sizes, may be useful.

5.1.3 Relations

Relations are sets that make use of paired/maplet elements (i.e., they define a relationship between two sets). Like sets, implementations may vary greatly, with the potential to increase efficiency of relation-specific operations.

Like sets, relations may benefit from compile-time computed properties: size of the domain, size of the range, totality, many-to-oneness, $O(1)$ access to the domain and range, and access to associated domain/range properties.

Relational Subtypes The notion of a relation is a broad classification of a set of pairs, where the domain, range, and relationship are not bound by any conditions. However, when generating code based on purely mathematical relations, it can be advantageous to use notions of totality, injectivity, surjectivity, etc. In particular, these named relational subtypes can be described with four properties: total on the domain, total on the range, one-to-many, and many-to-one.

The below table converts conventional notation into these four subtypes:

Relational Subtype	Properties			
	T	TR	1-	1-R
Relation				✓
Partial Function			✓	
Partial Injection			✓	✓
Surjective Relation		✓		
		✓		✓
Partial Surjection		✓	✓	
		✓	✓	✓
Total Relation	✓			
	✓			✓
Total Function	✓		✓	
Total Injection	✓		✓	✓
Total Surjective Relation	✓	✓		
	✓	✓		✓
Total Surjection	✓	✓	✓	
Bijection	✓	✓	✓	✓

Table 3: Conventional mathematical notation of relational subtypes decomposed into four properties: domain totality (T), range totality (TR), domain one-to-manyness (1-), and range one-to-manyness (1-R)

Subtype Properties Properties that can affect code generation may involve:

- New properties after applying operations on sets/relations
- Size of result
- Validity of operations
- Deterministic nature of operations
- Super/subset properties
- Relational identities

Below is a list of the impact of these properties.

Domain Totality:

- $R[S] \neq \emptyset$
- $R(S)$ is always valid
- R^{-1} is TR
- $|R| \geq \text{dom}(R)$

Range Totality:

- R^{-1} is T

- $|R| \geq \text{ran}(R)$

Domain One-to-manyness:

- $R(\{e\})$ is deterministic
- $|R[\{e\}]| = 1$
- $|R[S]| \leq |S|$
- $|R| \leq \text{dom}(R)$

Range One-to-manyness:

- $|R| \leq \text{ran}(R)$

5.1.4 Bags

Bags (or Multisets) are a refinement of relations wherein, for a bag with element type T , the bag is a relational function from $T \rightarrow \mathbb{Z}^+$, denoting the count of items in a collection. Once the count of an item reaches 0, the item is removed from the bag.

Bags must be many-to-one and carry specialized behaviours for conventional relation operations. For example, relational image ($R[B]$) with a bag argument will now return a bag, where the number of occurrences of the resulting item replaces what would otherwise be a regular set of those items.

Additional fields to record the size of the bag (sum of the range) may prove useful.

5.1.5 Sequences

Sequences are yet another refinement of relations where a sequence with element type T represents a relational function from $\mathbb{N} \rightarrow T$ that is total on some defined bounds.

The dense nature of sequence domains, along with knowledge of sequence sizes, may open up further data type refinements.

5.2 Data Type Operations

5.2.1 Primitives

5.2.2 Sets

5.2.3 Relations

5.2.4 Bags

5.2.5 Sequences

6 Implementation Specification

6.1 Abstract Data Types

6.2 Atomic Operations

6.3 Optimization Frontend

6.4 Optimization Backends

7 AST Lowering and Optimization Rules

Below is a list of rewrite rules for key abstract data types, syntactic sugar, and some builtin functions. Phases are intended to be executed in order; the post-condition of one phase serves as the pre-condition for the next.

7.1 Syntactic Sugar for Bags

$R[B] \rightsquigarrow R^{-1} \circ B$	(Bag Image)
$S \cup T \rightsquigarrow \llbracket x \cdot x \in \text{dom}(S) \cup \text{dom}(T) \wedge n = \max(S[x] \cup T[x]) \mid x \mapsto n \rrbracket$	(Bag Predicates - Union)
$S \cap T \rightsquigarrow \llbracket x \cdot x \in \text{dom}(S) \cap \text{dom}(T) \wedge n = \min(S[x] \cup T[x]) \wedge n > 0 \mid x \mapsto n \rrbracket$	(Bag Predicates - Intersection)
$S + T \rightsquigarrow \llbracket x \cdot x \in \text{dom}(S) \cup \text{dom}(T) \wedge n = \text{sum}(S[x] \cup T[x]) \mid x \mapsto n \rrbracket$	(Bag Predicates - Sum)
$S - T \rightsquigarrow \llbracket x \cdot x \in \text{dom}(S) \wedge n = S(x) - (T \cup \llbracket x \mapsto 0 \rrbracket)(x) \wedge n > 0 \mid x \mapsto n \rrbracket$	(Bag Predicates - Difference)

7.2 Syntactic Sugar for Sequences

$L \mathbin{++} M \rightsquigarrow L \cup \{x \mapsto y \cdot x \mapsto y \in M \mid x + \max(\text{dom}(L)) \mapsto y\}$	(Concatenation)
$\text{flatten}(X) \rightsquigarrow \text{foldL}(+, [], X)$	(Flatten)
$\text{foldL}(f, e, X) \rightsquigarrow \text{foldL}(f, f(e, \text{first}(X)), \text{tail}(X))$	(Fold Left Associative)
$\text{foldL}(f, e, []) \rightsquigarrow e$	(Fold Left Associative - Empty)
$\text{first}(X) \rightsquigarrow X(0)$	(First)
$\text{tail}(X) \rightsquigarrow \{i \mapsto x \cdot i \mapsto x \in X \wedge i \neq 0 \mid i - 1 \mapsto x\}$	(Tail)
$\text{append}(X, y) \rightsquigarrow X \cup \{\max(\text{dom}(X)) \mapsto y\}$	(Append)
$\text{prepend}(X, y) \rightsquigarrow \{i \mapsto x \cdot i \mapsto x \in X \mid i + 1 \mapsto x\} \cup 0 \mapsto y$	(Prepend)

7.3 Builtin Functions

$f(x) := E \rightsquigarrow f := f \oplus \{x \mapsto E\}$	(Functional Override)
$card(S) \rightsquigarrow \sum x \cdot x \in S \mid 1$	(Cardinality)
$dom(R) \rightsquigarrow \{x \mapsto y \cdot x \mapsto y \in R \mid x\}$	(Domain)
$ran(R) \rightsquigarrow \{x \mapsto y \cdot x \mapsto y \in R \mid y\}$	(Range)
$R \oplus Q \rightsquigarrow Q \cup (dom(Q) \triangleleft R)$	(Override)
$S \triangleleft R \rightsquigarrow \{x \mapsto y \cdot x \mapsto y \in R \wedge x \in S \mid x \mapsto y\}$	(Domain Restriction)
$S \triangleleft R \rightsquigarrow \{x \mapsto y \cdot x \mapsto y \in R \wedge x \notin S \mid x \mapsto y\}$	(Domain Subtraction)
$R \triangleright S \rightsquigarrow \{x \mapsto y \cdot x \mapsto y \in R \wedge y \in S \mid x \mapsto y\}$	(Range Restriction)
$R \triangleright S \rightsquigarrow \{x \mapsto y \cdot x \mapsto y \in R \wedge y \notin S \mid x \mapsto y\}$	(Range Subtraction)
$size(S) \rightsquigarrow \sum x \mapsto n \in S \cdot n$	(Bag Size)

7.4 Comprehension Construction

Intuition All set-like variables and literals are decomposed into set comprehensions.

Post-condition

- All terms with a set-like type (relations, bags, sets, sequences, etc.) must be in comprehension form.

$R(x) \rightsquigarrow choice(R[x])$	(Functional Image ^a)
$R[S] \rightsquigarrow \{x \mapsto y \cdot x \mapsto y \in R \wedge x \in S \mid y\}$	(Image)
$x \mapsto y \in S \times T \rightsquigarrow x \in S \wedge y \in T$	(Product)
$x \mapsto y \in R^{-1} \rightsquigarrow y \mapsto x \in R$	(Inverse)
$x \mapsto y \in (Q \circ R) \rightsquigarrow x \mapsto z \in Q \wedge z' \mapsto y \in R \wedge z = z'$	(Composition)

^aMay fail if R is not total. *choice* is nondeterministic.

$S \cup T \rightsquigarrow \{x \cdot x \in S \vee x \in T\}$	(Predicate Operations - Union)
$S \cap T \rightsquigarrow \{x \cdot x \in S \wedge x \in T\}$	(Predicate Operations - Intersection)
$S \setminus T \rightsquigarrow \{x \cdot x \in S \wedge x \notin T\}$	(Predicate Operations - Difference)
$x \in \oplus(E \mid P) \rightsquigarrow P \wedge x = E$	(Membership Collapse ^a)

^aRule only matches inside the predicate of a quantifier. Explicitly enumerating all matches for all quantification types and predicate cases (ANDs, ORs, etc.) would require too much boilerplate. *x* must be bound by the encasing quantifier. The $\oplus$ operator represents any quantifier that returns a set-like type (ex. generalized union/intersection, set comprehension, relation comprehension).

7.5 Disjunctive Normal Form

Intuition All quantifier predicates are expanded to DNF (i.e. $\wedge$ -operations nested within top-level $\vee$ -operations).

Notes All matches of this phase occur only inside quantifier predicates.

Post-condition

- All terms with a set-like type (relations, bags, sets, sequences, etc.) must be in comprehension form.
- Quantifier predicates are in disjunctive normal form - top level or-clauses with inner and-clauses.
- If no $\vee$ operators exist within a quantifier predicate, the predicate must only contain $\wedge$ operators.

$$\begin{aligned}
 x_1 \wedge \dots \wedge (x_i \wedge x_{i+1}) \wedge \dots &\rightsquigarrow x_1 \wedge \dots \wedge x_i \wedge x_{i+1} \wedge \dots && \text{(Flatten Nested } \wedge \text{)} \\
 x_1 \vee \dots \vee (x_i \vee x_{i+1}) \vee \dots &\rightsquigarrow x_1 \vee \dots \vee x_i \vee x_{i+1} \vee \dots && \text{(Flatten Nested } \vee \text{)} \\
 \neg \neg x &\rightsquigarrow x && \text{(Double Negation)} \\
 \neg(x \vee y) &\rightsquigarrow \neg x \wedge \neg y && \text{(Distribute De Morgan - Or)} \\
 \neg(x \wedge y) &\rightsquigarrow \neg x \vee \neg y && \text{(Distribute De Morgan - And)} \\
 x \wedge (y \vee z) &\rightsquigarrow (x \wedge y) \vee (x \wedge z) && \text{(Distribute } \wedge \text{ over } \vee \text{)}
 \end{aligned}$$

7.6 Or-wrapping

Intuition Restructuring quantifier predicates with top-level ORs for later rules.

Post-condition

- All terms with a set-like type (relations, bags, sets, sequences, etc.) must be in comprehension form.
- Quantifier predicates are in disjunctive normal form - top level or-clauses with inner and-clauses.

$$\{E \cdot \bigwedge P_i\} \rightsquigarrow \{E \cdot \bigvee \bigwedge P_i\} \quad \text{(Or-wrapping } ^a \text{)}$$

^aTo simplify the matching process later on, we wrap every top-level AND statement (which is guaranteed to be a ListOp by the dataclass field type definition) with an OR.

7.7 Generator Selection

Intuition All nested ANDs are placed in a tree structure to better suit loop lowering.

Post-condition

- All terms with a set-like type (relations, bags, sets, sequences, etc.) must be in comprehension form.
- Each top-level or-clause within quantification predicates must have one selected ‘generator’ predicate (GeneratorSelectionPredicate - GSP) of the form $x \in S$ that loops over its bound dummy variable x .
- All conjunctive clauses are wrapped in either a GSP or CombGSP structure.
- Quantifier predicates are in disjunctive normal form (with GSP replacing AND structures) - top level or-clauses with inner and-clauses.

Notes All matches of this phase occur only inside quantifier predicates. GSP is a tuple of form (generator chain, predicates) that can be flattened to a conjunctive clause. A CombGSP is a triple of form (shared generator, disjunctive children predicates/generators, predicates) that can likewise be flattened into a conjunctive clause. We keep the structure of these tuples distinct to ease the rewriting process.

Brainstorming selection heuristics Conditions to consider:

- Prefer composition chains in case of nested loops (ex. $x \mapsto y \in R \wedge y \mapsto z \in Q$, but what about renaming? - $x \mapsto y \in R \wedge y' \mapsto z \in Q \wedge y = y'$). Perhaps this could just be a preference for relations over sets if we see a nested quantifier.
- Choose most common generator among a list of or-clauses (allows for greater simplification later on)
- Choose smallest generator (by set size). This information may not always be accessible (ex. if a set is created by reading values from standard input). Functions may need to choose this on a case-by-case basis (ie. one function call could have two args, with a small set on the left arg and a large set on the right arg. But what if the sizes are reversed later in the code? We've already statically lowered it).
- When should constant folding happen? Equality substitution necessary for this?
- This may need to work with nesting considerations.
- What if we try moving these optimizations to *loop* structures far later in the pipeline?

$$\bigwedge P_i \rightsquigarrow GSP(\bigwedge P_{g_i}, \bigwedge_{P_i \notin P_g} P_i) \quad (\text{GSP Wrapping } ^a)$$

$$GSP(x \wedge xs, P) \vee GSP(x \wedge ys, Q) \rightsquigarrow CombGSP(x, GSP(xs, P) \vee GSP(ys, Q)) \quad (\text{Nested Generator Selection } ^b)$$

^aGSP is a tuple-like structure, where all entries can be flattened into an AND. The LH term must occur inside a quantifier's predicate - one generator per top-level or-clause. Chained generators may be necessary to allow for nested loop operations (ie. non-top-level clauses may require generators). P_g is a list of chained generators, specific set membership clauses (of form $x \in S$) distinguished from the rest of $\bigwedge P_i$. Currently, the selection of P_g is arbitrary (and thus the rewrite system is not confluent), but heuristics may be added later to choose better generators.

^b*free* is a function that returns true when variables in the predicate are defined (either bound by the current generator or bound prior to the current statement)

7.8 GSP to Loop

Intuition Start lowering expressions into imperative-like loops.

Post-condition

- All quantifiers are transformed into *loop* structures.
- *loop* predicates are of the form $x \in S \wedge \bigwedge P_i$.
- All *loop* predicates have an assigned generator of the form $x \in S$.

$$\oplus E \mid P \rightsquigarrow \begin{array}{l} a := \text{identity}(\oplus) \\ \mathbf{loop} \ P : \\ a := \text{accumulate}(a, E) \end{array} \quad (\text{Quantifier Generation } ^a)$$

^a $\oplus$ works for any quantifier (but not $\forall$ and $\exists$). The identity and accumulate functions are determined by the realized $\oplus$. For example, if $\oplus = \sum$, the identity is 0 and accumulate is addition.

$$\mathbf{loop} \bigvee P_i : \text{body} \rightsquigarrow \begin{array}{l} \mathbf{loop} \ P_0 \\ \text{body} \\ \mathbf{loop} \ P_1 \wedge \neg P_0 \\ \text{body} \\ \mathbf{loop} \ P_2 \wedge \neg \bigvee_{i < 2} P_i \\ \text{body} \end{array} \quad (\text{Top-level Or-Loop})$$

$$\mathbf{loop} \ GSP(g \wedge gs, P) \text{ body} \rightsquigarrow \begin{array}{l} \mathbf{loop} \ \text{SingleGSP}(g, \text{free}(P)) \\ \mathbf{loop} \ GSP(gs, \text{bound}(P)) \\ \text{body} \end{array} \quad (\text{Chained GSP Loop } ^a)$$

^a free considers defined variables at this point in time.

$$\mathbf{loop} \ GSP([], P) \text{ body} \rightsquigarrow \mathbf{if} \ P \ \mathbf{then} \ \text{body} \quad (\text{Empty GSP Loop})$$

$$\mathbf{loop} \ \text{CombGSP}(g, gs, P) \text{ body} \rightsquigarrow \begin{array}{l} \mathbf{loop} \ \text{SingleGSP}(g, P) \\ \mathbf{loop} \ gs_0 \\ \text{body} \\ \mathbf{loop} \ gs_1 \\ \text{body} \\ \dots \end{array} \quad (\text{Combined GSP Loop})$$

7.9 Relational Subtyping Loop Simplification

Intuition Simplify unnecessary iteration structures into direct accesses.

Post-condition

- All quantifiers are transformed into *loop* structures.
- *loop* predicates are of the form $x \in S \wedge \bigwedge P_i$.
- All *loop* predicates have an assigned generator of the form $x \in S$.

No change from previous, but the number of loop structures should not increase.

$$x \in \text{dom}(R) \rightsquigarrow \text{True}$$

(Total membership elimination ^a)

$$x \in \text{ran}(R) \rightsquigarrow \text{True}$$

(Surjective membership elimination ^b)

^a R is total and x satisfies the type of R 's domain. Currently unused since it may eliminate some generators - move higher in the list?

^b R is surjective and x satisfies the type of R 's range. Currently unused since it may eliminate some generators - move higher in the list? Or

loop *SingleGSP*($x \mapsto y \in R, x = a \wedge P$)
body

$\rightsquigarrow$ **if** $a \in \text{dom}(R)$ **then**
 loop *SingleGSP*($y \in R[a], P[x := a]$)
 body

(Concrete Domain Image)

loop *SingleGSP*($x \mapsto y \in R, y = a \wedge P$)
body

$\rightsquigarrow$ **if** $a \in \text{ran}(R)$ **then**
 loop *SingleGSP*($x \in R^{-1}[a], P[y := a]$)
 body

(Concrete Range Image ^a)

^aRequires efficient lookups for *ran*, inverse, and image

loop *SingleGSP*($y \in R[x], P$)
body

$\rightsquigarrow$ **if** $P[y := R(a)]$ **then**
 body[$y := R(a)$]

where *free*(x) and R is many-to-one

(Single Element Loop)

loop *SingleGSP*($x \in \{y\}, P$)
body

$\rightsquigarrow$ **if** $P[x := y]$ **then**
 body[$x := y$]

(Singleton Membership Elimination)

7.10 Loop Code Generation

Intuition Eliminate all intermediate loop structures.

Post-condition

- AST is in imperative code style (for loops, if statements, etc).
- All *loop* and quantification constructs have been eliminated.
- Some variables may not be defined.

loop *SingleGSP*($P_g, \bigwedge P_i$)
body

$\rightsquigarrow$ **if** $\bigwedge_{\text{free}(P_i)} P_i$ **then**
 for P_g **do**
 if $\bigwedge_{\text{bound}(P_i)} P_i$ **then**
 body

(Conjunct Conditional ^a)

^aFunction *free* returns clauses in P that contain only free + defined variables. *bound* returns the clauses that contain the bound variable x or undefined variables. P_g is the selected generator.

7.11 Replace and Simplify

Intuition Eliminate all undefined variables (with implicit $\exists$ quantifiers).

Post-condition

- AST is in imperative code style (for loops, if statements, etc).
- All variables are defined.

$$\begin{array}{ccc} \text{if } Identifier(x) = E \wedge P \text{ then} & & \\ \text{body} & \rightsquigarrow & \text{if } P[x := E] \text{ then} \\ & & \text{body}[x := E] \end{array} \quad (\text{Equality Elimination } ^a)$$

^a x is an undefined, unbound variable in the current scope. E is an expression that does not contain x . Simplify resulting booleans.

$$\begin{array}{ll} P \wedge True \rightsquigarrow P & (\text{Simplify And-true}) \\ P \wedge False \rightsquigarrow False & (\text{Simplify And-false}) \\ P \vee True \rightsquigarrow True & (\text{Simplify Or-true}) \\ P \vee False \rightsquigarrow P & (\text{Simplify Or-false}) \\ Statement(Statement(x)) \rightsquigarrow Statement(x) & (\text{Flatten Nested Statements}) \end{array}$$